INF8175 - IA : méthodes et algorithmes

Projet - Agent intelligent pour le jeu Hex

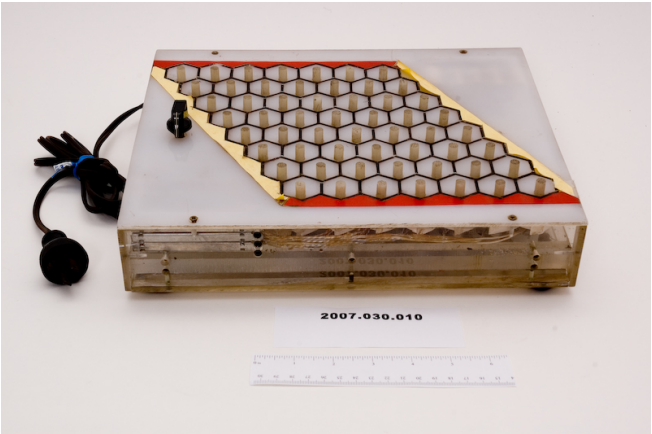

Figure 1 – Machine de jeu Hex par Claude et Elizabeth Shannon, 1953. [1]

Par Mattias Vinet-Larouche (2212749)
et Olivier Thabet (2294559)
11 mars 2026

Table des matières

1 Introduction **3**

2 Heuristique **3**
2.1 Principe 3

3 Stratégies exactes **3**
3.1 Ponts virtuels 3
3.2 Gabarits de bord 3
3.3 Filtrage des ponts virtuels adverses 3
3.4 Ouverture au centre 3

4 Méthodologie et évolution de l'agent **4**
4.1 Agent 1 : MiniMax de base 4
4.2 Agent 2 : MiniMax optimisé 4
4.3 Agent 3 : MiniMax + initialisation à chaud 4
4.4 Agent 4 : (`fullheuristicvolt`) — Agent final 4

5 Résultats **4**
5.1 Précision 4
5.2 Performance 4

6 Pistes d'améliorations et conclusion **4**

1 Introduction

Ce rapport décrit le développement d'un agent intelligent pour le jeu de Hex sur un plateau 14×14, sans règle de l'échange (*swap*). Le jeu de Hex est un jeu de plateau déterministe, à deux joueurs, à tour par tour, à somme nulle et à information parfaite. Le joueur Rouge (R) cherche à relier le bord supérieur au bord inférieur, tandis que le joueur Bleu (B) cherche à relier le bord gauche au bord droit. Une propriété fondamentale de Hex est qu'il ne peut pas y avoir de match nul : un gagnant existe toujours.

Notre démarche de développement a été incrémentale : nous avons commencé par un agent MiniMax simple utilisant le modèle électrique de Shannon comme heuristique, avant d'ajouter progressivement des optimisations algorithmiques (élagage alpha-bêta, cache, matrices creuses), des connaissances stratégiques (ponts virtuels, gabarits de bord), et enfin un contrôle adaptatif du temps (approfondissement itératif, stratégie hybride).

2 Heuristique

2.1 Principe

Le jeu de Hex est un jeu de connexion. Comme Shannon l'a fait avec sa machine de jeu Hex [1], nous modélisons le plateau de jeu comme un réseau électrique. Pour résoudre ce réseau, nous utilisons l'analyse nodale, qui est une méthode efficace pour déterminer la tension à chaque noeud du réseau. On peut utiliser cette approche car tous les éléments du réseau sont des résistances linéaires. On crée donc la matrice d'admittance du réseau G de taille $n^2 \times n^2$, avec n la taille du plateau.

Pour chaque connexion entre les cases (adjacentes), on définit la conductance g_{ij} entre les cases i et j. Si les cases sont occupées par deux de nos pièces, on attribue une conductance élevée (1000) pour représenter une connexion forte. À l'inverse, si les cases sont occupées par des pièces adverses, on attribue une conductance faible (0). Essentiellement, la connectivité est nulle. Pour les autres cas, on attribue une conductance intermédiaire (1).

3 Stratégies exactes

3.1 Ponts virtuels

Un pont virtuel est une formation où deux pierres alliées sont reliées par deux chemins intermédiaires distincts. Cette connexion est **virtuellement garantie** : si l'adversaire joue un intermédiaire, on joue l'autre. Il ne s'agit pas d'une heuristique mais d'une vérité mathématique sur le jeu de Hex.

Dans notre modèle électrique, les ponts virtuels alliés sont modélisés par une connexion de forte conductance entre les deux pierres alliées. La conductance dépend de l'état des intermédiaires :

- Deux intermédiaires vides : conductance = 400
- Un intermédiaire déjà allié : conductance = 800
- Cas partiel : conductance = 100

3.2 Gabarits de bord

Les gabarits de bord sont des configurations prouvées qui garantissent qu'une pierre proche d'un bord peut s'y connecter, indépendamment du jeu adverse. Nous en modélisons deux types :

- **Distance 2** (rangée $r = 1$ ou $r = n - 2$) : bonus de conductance de 500 vers le bord
- **Distance 3** (rangée $r = 2$ ou $r = n - 3$) : bonus de conductance de 200 si le couloir intermédiaire est libre

3.3 Filtrage des ponts virtuels adverses

Lorsque l'adversaire possède un pont virtuel intact, jouer dans ses intermédiaires est généralement inefficace : il répondra en jouant l'autre. Notre agent filtre donc ces cases et n'y joue que si aucune autre option n'est disponible.

3.4 Ouverture au centre

En tout début de partie (moins de 2 pierres posées), l'agent contourne complètement le calcul heuristique et joue directement une case centrale parmi une liste de cinq positions prioritaires : le centre exact du plateau, puis quatre décalages en mouvement de cavalier autour de celui-ci. Cette alternance à

l'heuristique permet d'économiser un appel coûteux au solveur électrique lorsque le résultat est connu d'avance : le centre et ses abords immédiats constituent les meilleures ouvertures sur Hex, indépendamment de l'état du plateau.

4 Méthodologie et évolution de l'agent

4.1 Agent 1 : MiniMax de base

Le premier agent utilise l'algorithme MiniMax avec élagage alpha-bêta à profondeur fixe $d = 2$. Le modèle électrique est construit avec une matrice dense NumPy de taille $n^2 \times n^2$, soit $196^2 = 38\,416$ entrées pour un plateau 14×14. Il n'y a aucun cache, aucune connaissance stratégique (ponts virtuels, gabarits de bord), et l'objet `GameStateHex` de Seahorse est manipulé directement à chaque appel, ce qui ajoute un surcoût important.

4.2 Agent 2 : MiniMax optimisé

La version V3 introduit trois améliorations majeures :

Géométrie précalculée : La structure du plateau (voisins, liens de pont virtuel) est calculée une seule fois dans `_get_geom`. Le plateau est représenté comme un vecteur `List[str]` au lieu d'un objet `GameStateHex`.

Matrice creuse : On abandonne la matrice dense au profit d'une matrice creuse COO/CSR, réduisant la mémoire et le temps de construction.

Cache LRU : Les résultats d'ampérage et d'évaluation sont mis en cache dans des `OrderedDict` à 50 000 entrées, avec une table de transposition `_tt` pour le MiniMax.

Des connaissances stratégiques sont introduites : ouverture au centre, détection des coups gagnants immédiats, et modélisation des ponts virtuels alliés dans le circuit électrique.

4.3 Agent 3 : MiniMax + initialisation à chaud

La V4 reprend la V3 MiniMax et ajoute une *initialisation à chaud* pour la résolution par gradient conjugué : la solution de tension calculée au coup précédent est réutilisée comme point de départ x_0, réduisant le nombre d'itérations nécessaires à la convergence (`CG_MAXITER = 40`).

4.4 Agent 4 : (`fullheuristicvolt`) — Agent final

5 Résultats

5.1 Précision

5.2 Performance

6 Pistes d'améliorations et conclusion

Références

[1] Claude Elwood Shannon et Mary Elizabeth (Moore) Shannon. *Hex (Hox)*. Object ID : 2007.030.010. MIT Museum. 1953. URL : https://mitmuseum.mit.edu/collections/object/2007.030.010 (visité le 15/03/2026).